

Lab5 实验报告

一、思考题

Thinking 5.1

如果通过 `kseg0` 读写设备，设备寄存器访问会经过 Cache，这会破坏 MMIO 设备访问的语义。设备寄存器不是普通内存，读写往往有副作用，并且要求立即生效。

可能的问题包括：

- 写操作可能只写入 Cache，没有真正到达设备寄存器，导致设备没有收到命令。
- 读操作可能读到 Cache 中的旧值，无法获得设备当前状态。
- Cache 的写回、合并、重排可能改变设备访问顺序。因此设备访问应通过不缓存的地址段，如 `kseg1`。

Exercise 5.4

`free_block(blockno)` 中 `blockno` 不能为 0，是因为磁盘块 0 是 boot 块，不属于普通文件系统可分配的数据块。文件系统布局中：

```
block 0: boot/reserved
block 1: super block
block 2..: bitmap blocks
之后：文件块、目录块、间接索引块
```

Thinking 5.2

相关定义在 `user/include/fs.h`：

```
#define BLOCK_SIZE PAGE_SIZE          // 4096B
#define FILE_STRUCT_SIZE 256
#define FILE2BLK (BLOCK_SIZE / sizeof(struct File))
#define NDIRECT 10
#define NINDIRECT (BLOCK_SIZE / 4)
#define MAXFILESIZE (NINDIRECT * BLOCK_SIZE)
```

1. 一个磁盘块最多能存储多少个文件控制块？

`struct File` 大小为 256B，一个磁盘块为 4KB：

```
4096 / 256 = 16
```

所以一个磁盘块最多存储 16 个文件控制块。

2. 一个目录下最多能有多少个文件？

目录本质上也是文件，它的数据块中存放 `struct File` 数组。单个文件最多有 $NINDIRECT = 4096 / 4 = 1024$ 个文件块，因此理论上一个目录最多能容纳：

1024 * 16 = 16384

即最多约 16384 个目录项。实际数量还会受磁盘总大小和已用块数量限制。

3. 单个文件最大为多大？

MAXFILESIZE = NINDIRECT * BLOCK_SIZE = 1024 * 4096 = 4MB

因此该文件系统支持的单个文件最大为 4MB。

Thinking 5.3

磁盘块缓存区从 `DISKMAP` 开始，最大覆盖 `DISKMAX` 字节：

```
#define DISKMAP 0x10000000
#define DISKMAX 0x40000000
```

$DISKMAX = 0x40000000 = 1GB$ ，因此在磁盘块缓存设计允许的范围内，内核支持的最大磁盘大小是 1GB。

若以 4KB 为一个块，则最大块数为：

1GB / 4KB = 0x40000000 / 0x1000 = 0x40000 = 262144 块

实验生成的 `fs.img` 通常只有 1024 个 4KB 块，即 4MB，这是实验镜像大小，不是缓存设计的上限。

Thinking 5.4

较重要的宏定义如下：

宏	含义	主要用途
BLOCK_SIZE	文件系统块大小，等于页大小 4KB	磁盘块读写、文件块映射
BLOCK_SIZE_BIT	一个 bitmap 块能表示的磁盘块数	计算 bitmap 块数量和对应 bitmap 块号
MAXNAMELEN	单个文件名最大长度	<code>struct File::f_name</code>
MAXPATHLEN	路径最大长度	<code>open/remove/path walk</code> 参数检查
NDIRECT	直接块指针数量，值为 10	<code>f_direct[]</code>

宏	含义	主要用途
NINDIRECT	间接索引项数量，值为 1024	限制文件最大块数
MAXFILESIZE	单个文件最大大小	ftruncate、write 越界检查
FILE_STRUCT_SIZE	struct File 固定大小 256B	保证磁盘格式稳定
FILE2BLK	一个块能容纳的 File 数量	目录遍历
FS_MAGIC	文件系统魔数	校验 super block
DISKMAP	fs server 中磁盘块缓存起始地址	disk_addr(blockno)
DISKMAX	块缓存可覆盖的最大磁盘大小	限制最大磁盘大小
SECT_SIZE	IDE 扇区大小 512B	IDE 读写单位
SECT2BLK	一个文件系统块对应的扇区数	blockno 转 secno
PTE_DIRTY	文件系统缓存页脏标记	判断是否需要写回磁盘
PTE_LIBRARY	共享页标记	fork 后共享 fd 和文件映射页

这些宏把“磁盘块大小、文件结构大小、缓存地址布局、IPC 映射权限”等约定集中表达出来，是 Lab5 文件系统能在用户态服务进程中运行的基础。

Thinking 5.5

fork 前后的父子进程会共享文件描述符和定位指针。

原因是文件描述符页和文件数据映射页带有 PTE_LIBRARY 标记。Lab4 的 fork() 中，普通可写页会被设置为 COW，而带 PTE_LIBRARY 的页会直接共享映射。因此父子进程看到的是同一个 struct Fd 页面，fd_offset 也共享。

可以编写如下程序验证：父进程打开文件后 fork，子进程读一部分内容使 offset 前进，父进程等待子进程退出后继续读。如果父进程读到的是子进程读完之后的位置，说明定位指针共享。

```
#include <lib.h>

int main() {
    int fd = open("/newmotd", O_RDONLY);
    if (fd < 0) {
        user_panic("open: %d", fd);
    }

    int r = fork();
    if (r < 0) {
        user_panic("fork: %d", r);
    }

    char buf[8];
    if (r == 0) {
        read(fd, buf, 4);
        exit();
    }
```

```
    }

    wait(r);
    read(fd, buf, 4);
    buf[4] = 0;
    debugf("parent reads after child: %s\n", buf);
    close(fd);
    return 0;
}
```

若父进程第二次读取不是从文件开头开始，而是从子进程读过的位置继续，就说明 fd 和 offset 被共享。

Thinking 5.6

File、Fd、Filefd 的作用如下：

结构体	所在位置	是否对应磁盘实体	作用
struct File	fs 镜像、fs server 内存、Filefd 快照	是	文件控制块，记录文件名、大小、类型、直接块、间接块等
struct Fd	用户进程文件描述符页	否	用户态打开文件的抽象描述符，记录设备类型、当前偏移、打开模式
struct Filefd	用户进程 fd 页，由 fs server 映射返回	部分对应	文件设备专用 fd，包含 Fd、fileid 和 File 的一份快照

File 是文件系统的核心磁盘结构，目录的数据块中存放的就是一组 File。普通文件的 f_direct 和 f_indirect 指向文件内容块；目录文件的内容块则存放子文件/子目录的 File。

Fd 是用户态通用文件描述符，不只用于普通文件，也可用于 console、pipe 等设备。fd_offset 保存当前读写位置。

Filefd 是普通文件设备对 Fd 的扩展。用户执行 open() 时，fs server 会把一个 Filefd 页面映射回用户进程，用户再通过 f_fileid 向 fs server 请求 map、dirty、close 等操作。

Thinking 5.7

图 5.9 主要展示了 init、文件服务进程 fs_serv 和普通用户进程 user_env 之间的创建与通信关系。

图中的实线箭头表示进程创建关系：init 通过 ENV_CREATE(user_env) 创建普通用户进程，通过 ENV_CREATE(fs_serv) 创建文件服务进程。二者都是用户态进程，只是职责不同：普通用户进程负责发起文件操作请求，fs_serv 负责维护文件系统并响应请求。

图中的竖直虚线表示各进程各自的执行时间线。fs_serv 被创建后，会先执行 serve_init() 和 fs_init()，初始化打开文件表、读取 super block、检查磁盘、读取 bitmap，然后进入循环等待用户进程的请求。

图中 IPC 区域内的箭头表示用户进程与文件服务进程之间的 IPC 通信。用户进程执行 open() 时，会依次调用 fd_alloc()、fsipc_open()、fsipc()，最终通过 ipc_send(fsreq) 将请求页发送给 fs_serv；fs_serv 处理完成后，再通过 ipc_send(dst_va) 把返回值或映射页面传回用户进程。

图中的 `wait_for_next_serv` 表示文件服务进程处理完一次请求后不会退出，而是继续等待下一次 IPC 请求。也就是说，文件系统作为长期运行的用户态服务存在，普通用户进程通过 IPC 调用它完成文件操作。这体现了 MOS 中用户进程、文件服务进程和内核基础机制分离的微内核式设计。

二、难点分析与实验体会

1. 核心难点

文件系统层次较多。 用户程序并不直接访问磁盘，而是先调用用户态库函数，再通过 IPC 请求 fs server；fs server 管理文件系统结构和块缓存；真正访问设备时再陷入内核，由内核完成 MMIO。

块号和地址容易混淆。 `filebno` 是文件内块号，`diskbno` 是磁盘块号，`disk_addr(blockno)` 得到的是 fs server 的块缓存地址。用户进程读写文件时使用的是 `fd2data(fd)` 对应的映射区。

目录也是文件。 目录的内容块中存放的是 `struct File` 数组，因此目录查找本质上是遍历目录文件的数据块，再遍历每个块中的文件控制块。

bitmap 语义反直觉。 本实验中 bitmap 位为 1 表示空闲，为 0 表示已使用；分配块要清 bit，释放块要置 bit。

dirty 与写回时机需要理解。 用户写文件只是修改了映射页，还要通过 `fsipc_dirty()` 通知 fs server 标脏；真正写回磁盘通常发生在 `file_flush()`、`file_close()` 或 `fs_sync()` 中。

2. 实验收获/技巧总结

- 理解了用户进程、文件服务进程、内核、磁盘设备之间的分工：用户进程发请求，fs server 维护文件系统，内核提供系统调用和设备访问，磁盘只接受底层读写命令。
- 感受到 MOS 的层次化设计和微内核思想：文件系统服务放在用户态，内核只保留 IPC、内存映射、进程管理、设备访问等基础能力。
- 掌握了通过页面映射减少数据复制的方式，例如 fs server 将 Filefd 页和文件数据页映射给用户进程。
- 熟悉了位运算在 bitmap 中的应用：`blockno & 0x1f` 等价于 `blockno % 32`。
- 学习了 C 语言 `volatile` 的意义：访问可能被硬件、内核或其他执行流改变的数据时，需要避免编译器把读写优化掉。

3. 实验注意事项

- 修改 `struct File` 字段后要及时 `dirty_fcb(f)`。
- 遍历目录时要跳过 `f_name[0] == '\0'` 的空目录项。
- 判断打开模式时应使用 `(fd->fd_omode & O_ACCMODE)`，不能直接用 `fd->fd_omode & O_RDONLY`。
- 新增文件系统请求时，要同时修改 `fsreq.h`、`fs/serv.c`、`user/lib/fsipc.c`，必要时还要修改 `user/lib/file.c` 和 `lib.h`。
- 设备寄存器访问不能走缓存地址，应使用不缓存映射，避免读到旧状态或写入不生效。

三、原创说明

本报告基于本人实验过程、课程 MOS Lab5 代码阅读与调试整理完成，个人笔记+AI整理，部分概念表述参考实验指导书和源码注释。